



FPGA Assignments Report

Assignment 1 and Assignment 2

Author: Bassel Mahmoud

Date: July 22, 2026

Contents

1	Introduction	1
2	Assignment 1: 3-Bit Counter	1
2.1	Objective	1
2.2	Implementation	1
2.3	Test Bench	1
2.4	Design Summary	1
2.5	Figures	2
2.6	Result	3
3	Assignment 2: NAND Gate	4
3.1	Objective	4
3.2	Implementation	4
3.3	Design Summary	4
3.4	Figures	5
3.5	Result	6
4	Conclusion	6

1 Introduction

This report summarizes the two FPGA assignments completed in the current workspace.

- Assignment 1 implements a 3-bit counter driven by a 1 Hz clock derived from the internal oscillator through a PLL and a clock divider.
- Assignment 2 implements a NAND gate by building an AND gate in hardware and inverting the result in software running on the Zynq processor.

2 Assignment 1: 3-Bit Counter

2.1 Objective

The assignment objective is to implement a 3-bit counter driven by a 1 Hz clock. The clock is generated from the internal oscillator 100 MHz, passed through a PLL to produce an 8 MHz signal, and then divided down to 1 Hz so that the counter transitions are easy to observe on the FPGA LEDs.

2.2 Implementation

The design is built from three simple blocks:

- `clk_wiz_0`: generates the 8 MHz clock from the board clock.
- `clk_div`: divides the 8 MHz clock down to a 1 Hz output.
- `counter`: increments a 3-bit register on each rising edge of the 1 Hz clock.

The top module connects the clock wizard output to the divider and then feeds the counter with the divided clock. Reset is active-low and clears the counter to zero.

2.3 Test Bench

The testbench is build on Clock Divider that divide the PLL CLK to 1 MHz instade of 1 Hz to simulate it reasonable time.

2.4 Design Summary

Table 1: Assignment 1 Summary

Item	Description
Clock source	Board clock into PLL
Intermediate clock	8 MHz PLL output
Final clock	1 Hz clock from divider
Counter width	3 bits
Reset behavior	Counter resets to zero on <code>rst_n=0</code>
Output	3-bit LED-visible count

2.5 Figures

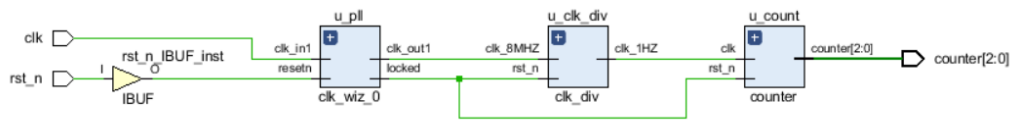


Figure 1: Assignment 1 elaborated design

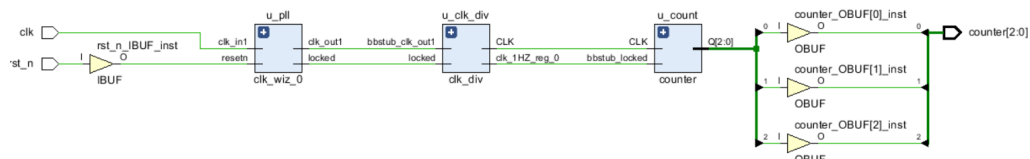


Figure 2: Assignment 1 synthesized design

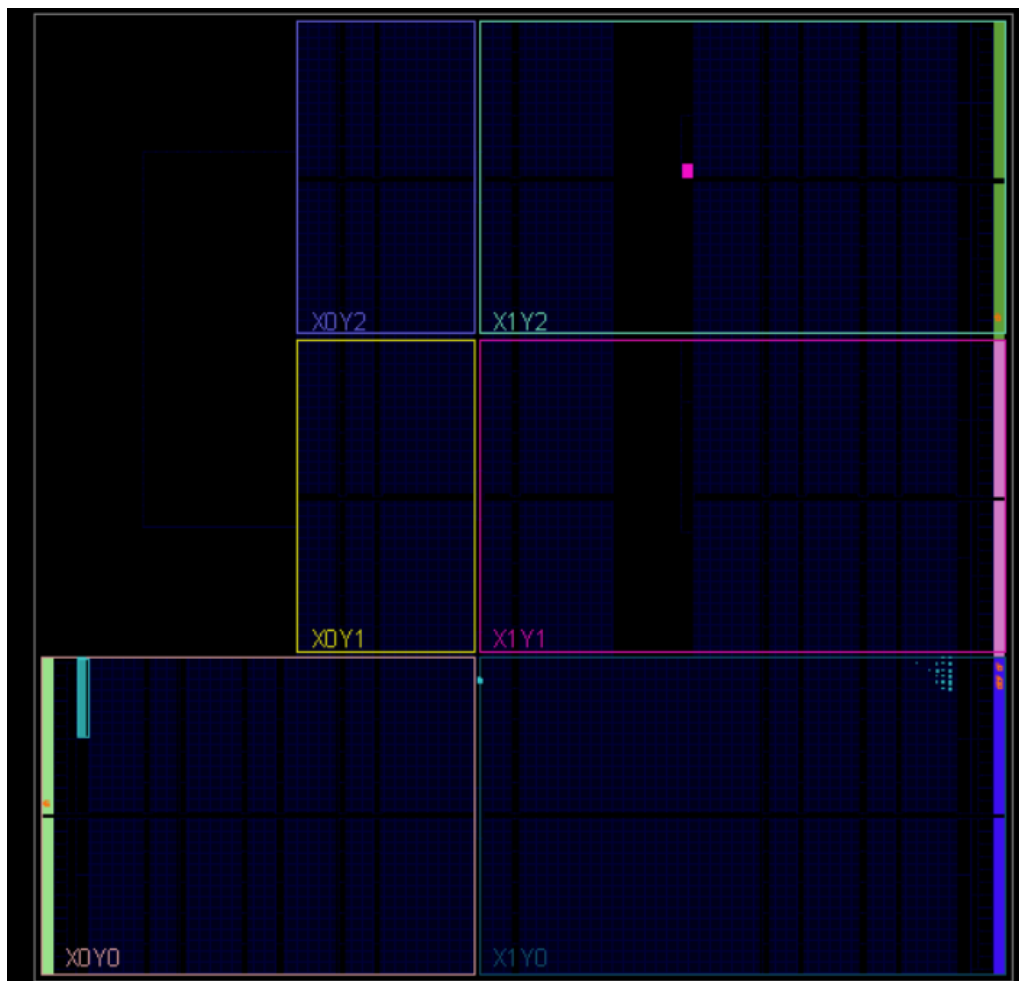


Figure 3: Assignment 1 implemented design

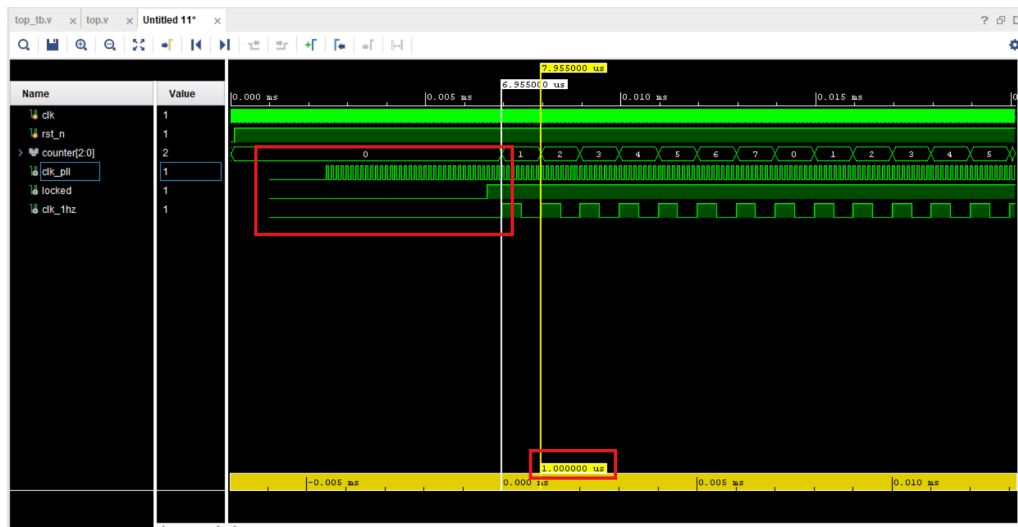


Figure 4: Assignment 1 simulation waveform

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 121.544 ns	Worst Hold Slack (WHS): 0.115 ns	Worst Pulse Width Slack (WPWS): 2.633 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 23	Total Number of Endpoints: 23	Total Number of Endpoints: 29

All user specified timing constraints are met.

Figure 5: Assignment 1 timing report

top.bit	22/07/2026 12:08 AM	BIT File	3,951 KB
---------	---------------------	----------	----------

Figure 6: Assignment 1 bitstream generation

2.6 Result

The design compiles successfully and produces a slow counter suitable for LED observation.

3 Assignment 2: NAND Gate

3.1 Objective

The assignment objective is to implement a NAND gate by designing an AND gate in hardware and then using a C program running on the Zynq processor to perform the inversion.

3.2 Implementation

The hardware part is a simple AND gate module:

```
module and_gate(  
    input a,b,  
    output c  
);  
    assign c = a&b;  
endmodule
```

The block design integrates the processing system, AXI GPIO blocks, and the custom logic. The processor reads or drives the hardware signals and performs the inversion in software to complete NAND behavior.

3.3 Design Summary

Table 2: Assignment 2 Summary

Item	Description
Hardware block	AND gate
Processor role	Invert the AND result in software
Platform	Zynq-based Vivado block design
Integration	AXI GPIO and processing system
Final logic	NAND behavior from hardware plus software inversion

3.4 Figures

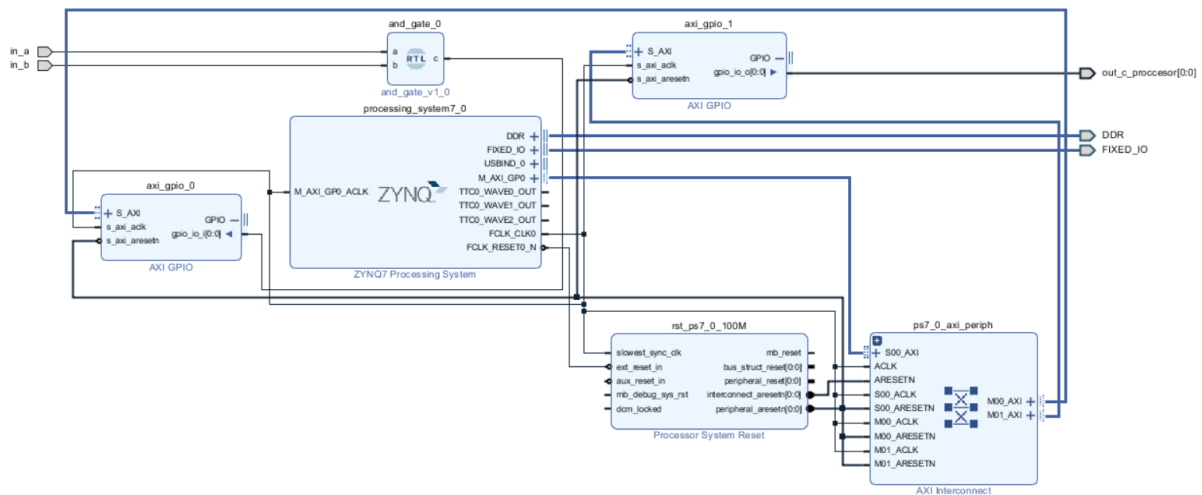


Figure 7: Assignment 2 block design

```
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include "xgpio.h"
#include "xparameters.h"

int main()
{
    init_platform();

    XGpio input , output;
    int a , y;
    XGpio_Initialize(&input,XPAR_AXI_GPIO_0_DEVICE_ID);
    XGpio_Initialize(&output,XPAR_AXI_GPIO_1_DEVICE_ID);
    XGpio_SetDataDirection(&input,1,1);
    XGpio_SetDataDirection(&output,1,0);

    while(1){
        a = XGpio_DiscreteRead(&input,1);
        y = ~a;
        XGpio_DiscreteWrite(&output,1,y);
    }
    cleanup_platform();
    return 0;
}
```

Figure 8: Assignment 2 inversion code

```

CDT Build Console [inv_logic]
23:14:09 **** Build of configuration Release for project inv_logic ****
make pre-build main-build
a9-linaro-pre-build-step
'Building file: ../src/inv_logic.c'
'Invoking: ARM v7 gcc compiler'
arm-none-eabi-gcc -Wall -O2 -c -fmessage-length=0 -MT"src/inv_logic.o" -mcpu=cortex-a9 -mfloat-abi=hard -I../inv_logic_bsp/ps7_cortexa9_0/include -WMD -MP -MF"src/inv_logic.d" -MT"src/inv_l
'Finished building: ../src/inv_logic.c'
'Building file: ../src/platform.c'
'Invoking: ARM v7 gcc compiler'
arm-none-eabi-gcc -Wall -O2 -c -fmessage-length=0 -MT"src/platform.o" -mcpu=cortex-a9 -mfloat-abi=hard -I../inv_logic_bsp/ps7_cortexa9_0/include -WMD -MP -MF"src/platform.d" -MT"src/platfor
'Finished building: ../src/platform.c'
'Building target: inv_logic.elf'
'Invoking: ARM v7 gcc linker'
arm-none-eabi-gcc -mcpu=cortex-a9 -mfloat-abi=hard -Wl,-build-id=none -specs=xilinx.spec -Wl,-T -Wl,../src/ldscript.ld -L../inv_logic_bsp/ps7_cortexa9_0/lib -o "inv_logic.elf" ../src/inv_log
'Finished building target: inv_logic.elf'
'Invoking: ARM v7 Print Size'
arm-none-eabi-size inv_logic.elf |tee "inv_logic.elf.size"
text      data      bss      dec      hex filename
20316     1176     22584     44076     ac2c inv_logic.elf
'Finished building: inv_logic.elf.size'
23:14:11 Build Finished (took 2s.131ms)

```

Figure 9: Assignment 2 successful build

3.5 Result

The system demonstrates the NAND function using a hardware AND gate and a software inversion step on the processor side.

4 Conclusion

Both assignments were completed successfully.

- Assignment 1 demonstrates clock generation, clock division, and a simple synchronous counter.
- Assignment 2 demonstrates basic Zynq-based hardware/software cooperation using an AND gate and software inversion.

The combined report keeps the documentation short, clear, and easy to review.